

1. Title of the Project

Morphed Face Image Detection Using PCA and Binary Classification

2. Objective of the Project

The objective of this project is to develop a machine learning based system capable of detecting morphed face images. Face morphing attacks combine two facial images to create a single manipulated image that can deceive biometric authentication systems.

This project aims to use **Principal Component Analysis (PCA)** for feature extraction and **binary classification algorithms** such as Support Vector Machine (SVM) to classify facial images as either original or morphed.

The main objectives of the project are:

- To study the concept of face morphing attacks and their impact on biometric systems.
- To preprocess facial image datasets for machine learning processing.
- To extract important facial features using Principal Component Analysis (PCA).
- To train machine learning models for detecting morphed images.
- To develop a system capable of identifying whether an image is original or morphed.

3. Problem Statement

Biometric authentication systems such as facial recognition are widely used in security applications including passport verification, border control, and identity authentication. However, these systems are vulnerable to **face morphing attacks**, where two facial images are digitally combined to produce a single manipulated image.

Such morphed images may allow multiple individuals to successfully authenticate using the same biometric identity, which poses serious security risks.

Therefore, there is a need for an automated system that can detect morphed face images and differentiate them from genuine facial images. This project proposes a machine learning based approach that uses **Principal Component Analysis (PCA)** and **binary classification algorithms** to identify morphed images and improve the reliability of biometric authentication systems.

4. Abstract

Face recognition systems are widely used in modern biometric authentication systems; however, these systems are vulnerable to morphing attacks where two facial images are blended to create a manipulated image capable of deceiving face recognition systems. This project proposes a machine learning based approach for detecting morphed face images using Principal Component Analysis (PCA) and binary classification algorithms. PCA is used to extract

important features from facial images and reduce the dimensionality of image data, enabling efficient processing. The extracted features are then used to train classification models such as Support Vector Machine (SVM) to identify whether a given face image is original or morphed. The proposed system processes facial images through several stages including **data collection, image preprocessing, PCA feature extraction, model training, and image classification**. The objective of this project is to develop an efficient morphed face detection system that enhances the security and reliability of biometric authentication systems.

5. Overall Architecture Diagram

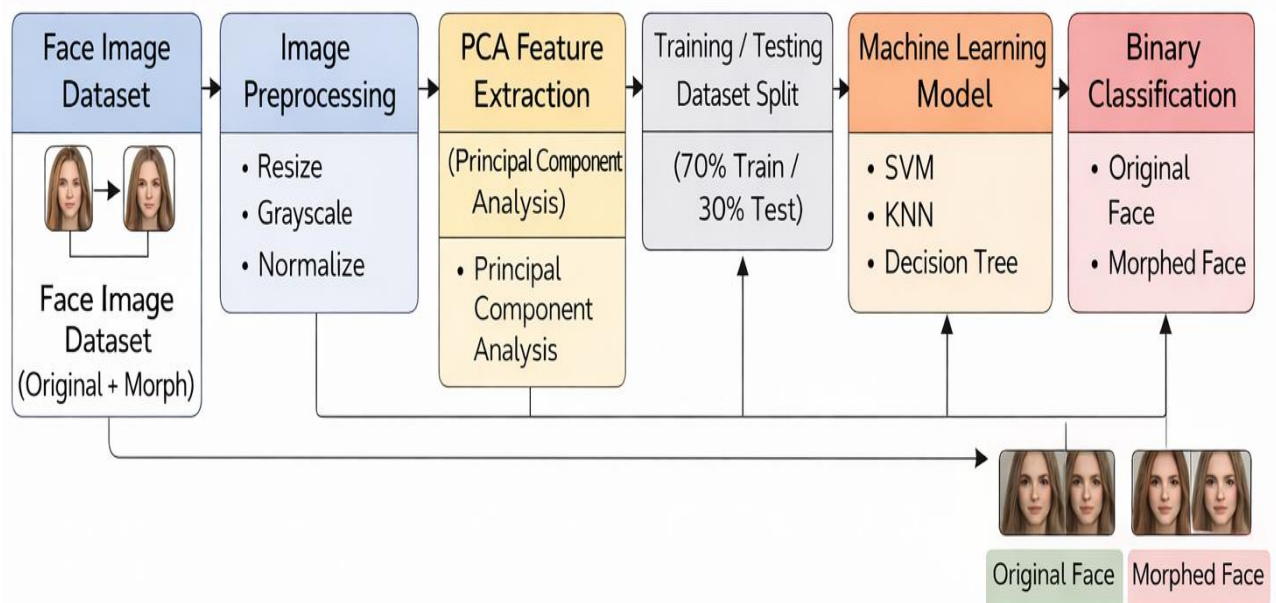


Fig 1: Overall Architecture of Morphed Face Detection System

6. Detailed Design Diagram (Module-Wise)

Module 1: Data Collection

In this module, facial image datasets containing both original and morphed images will be collected. The dataset will be obtained from online sources such as Kaggle. The images will be organized into two categories: original face images and morphed face images.

Each image will be labelled appropriately to indicate whether it belongs to the original or morphed class. This dataset will serve as the input for further processing in the system.

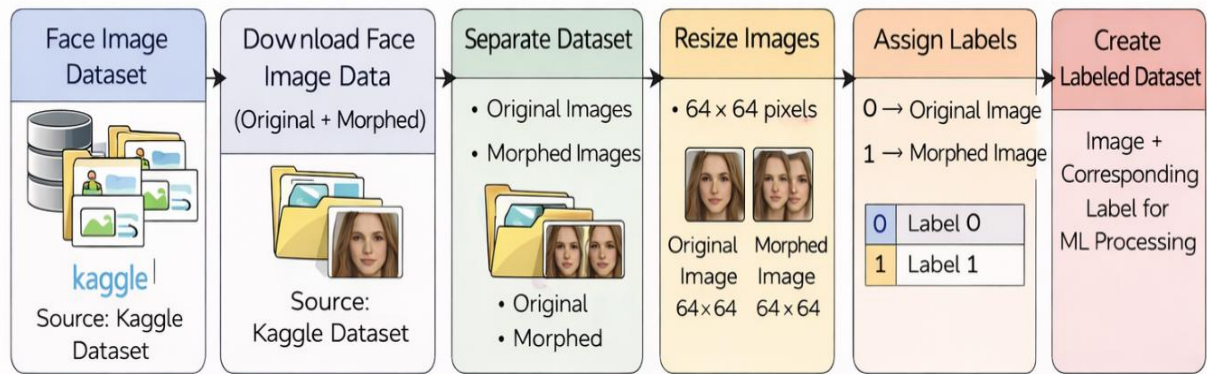


Fig 2: Module 1 – Data Collection Architecture

Module 2: Image Preprocessing

In this module, the collected facial images will be pre-processed to make them suitable for machine learning algorithms.

The preprocessing steps include:

- Resizing all images to a uniform size (64×64 pixels)
- Converting images to grayscale format
- Converting images into numerical pixel arrays
- Normalizing pixel values

These preprocessing steps will help reduce computational complexity and prepare the dataset for feature extraction.

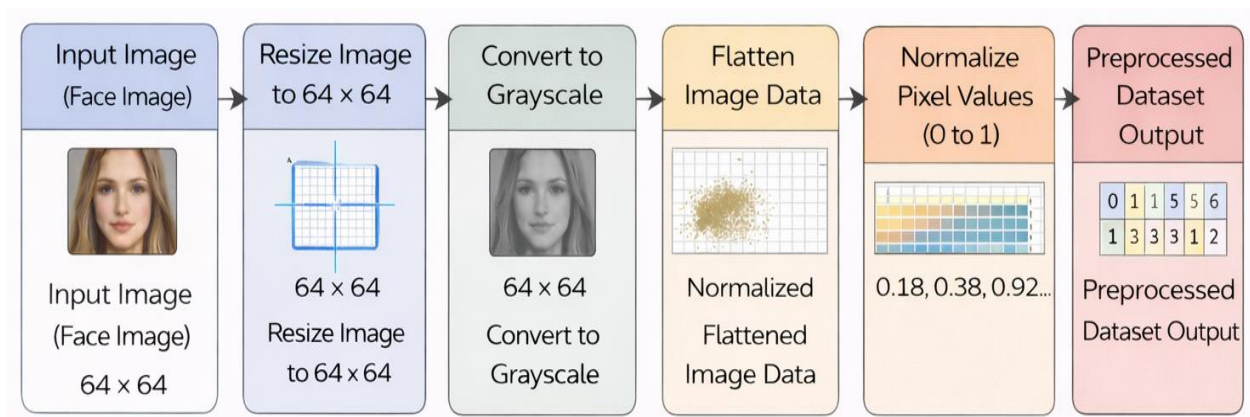


Fig 3: Module 2 – Image Preprocessing Architecture

Module 3: Feature Extraction using PCA

In this module, **Principal Component Analysis (PCA)** will be used to extract important features from the facial images.

PCA reduces the dimensionality of the dataset by identifying the principal components that represent the most important patterns in the data. These components capture the essential facial features while reducing redundant information.

The extracted PCA features will then be used as input for machine learning models.

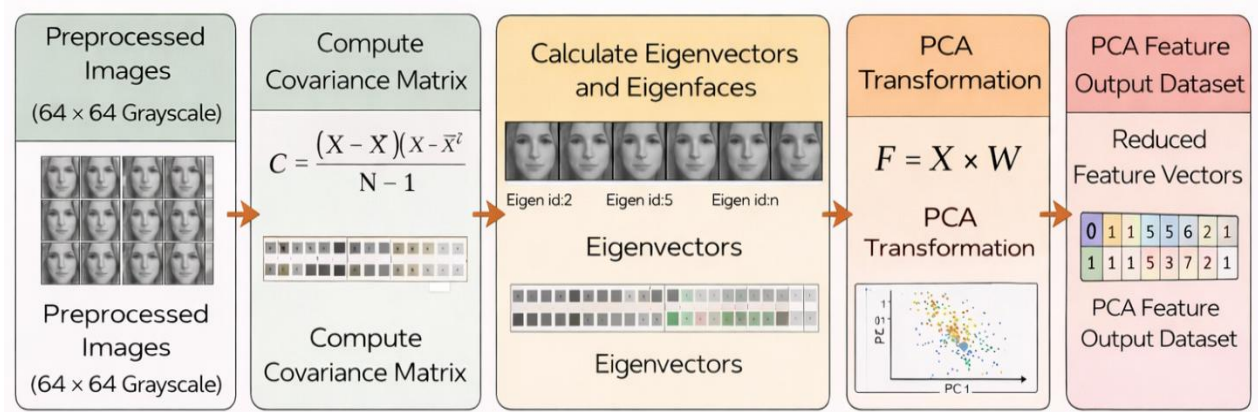


Fig 4: Module 3 – PCA Feature Extraction Architecture

Module 4: Model Training

In this module, machine learning algorithms will be trained using the PCA extracted features.

The dataset will be divided into **training and testing sets**. Various classification algorithms such as:

- Support Vector Machine (SVM)
- K-Nearest Neighbor (KNN)
- Decision Tree
- Logistic / Linear models

will be trained to learn the patterns that differentiate original and morphed images.

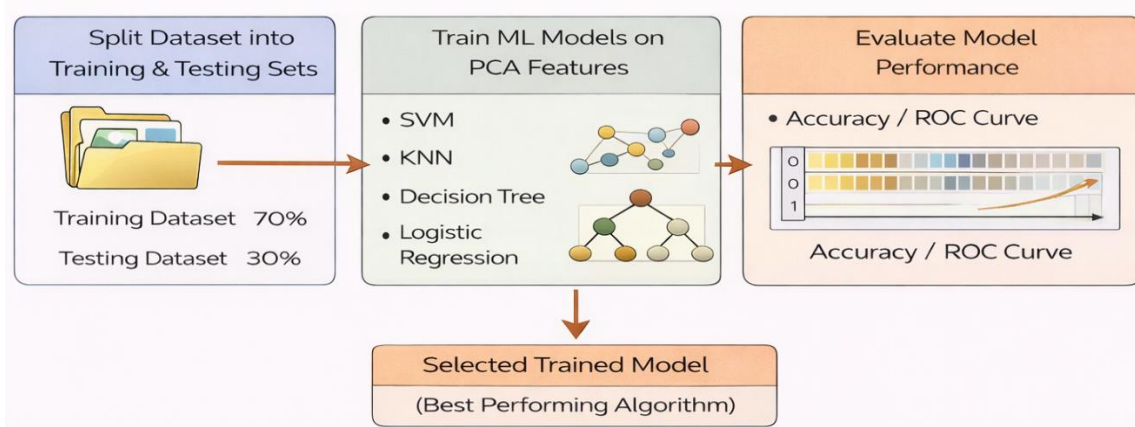


Fig 5: Module 4 – Model Training Architecture

Module 5: Image Classification

In this module, the trained machine learning model will be used to classify new facial images.

When a new image is provided to the system, it will undergo the same preprocessing and PCA feature extraction steps. The extracted features will then be passed to the trained classifier, which will predict whether the image is **original or morphed**.

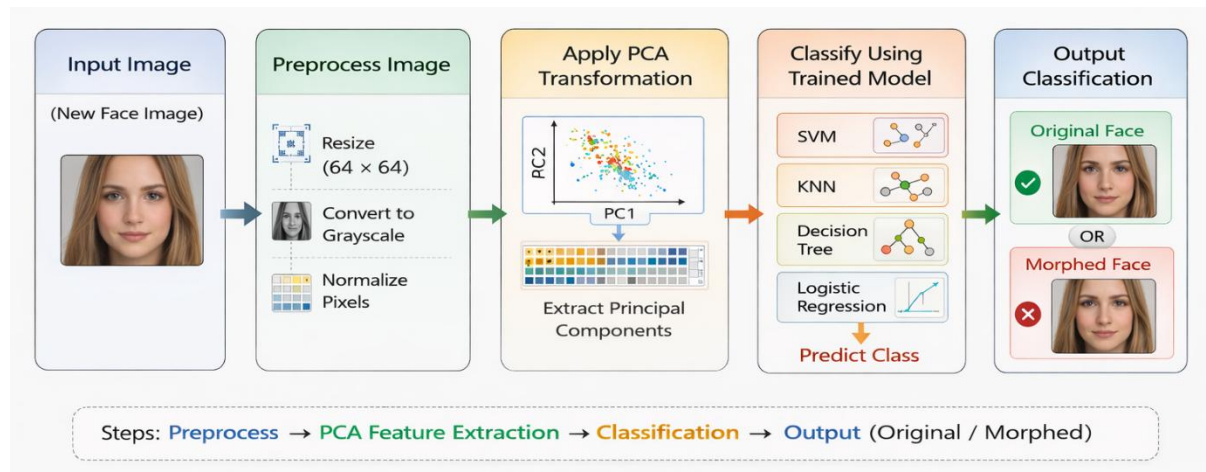


Fig 6: Module 5 – Image Classification Architecture

7. Dataset Description

In this project, I use the “**Real and Fake Face Detection**” dataset from **Kaggle**, developed by the **Computational Intelligence and Photography Lab (CIPLab)**, **Yonsei University, South Korea**. The dataset consists of **2,041 facial images**, including **1,081 original (real) images** and **960 morphed (fake) images**. These morphed images are high-quality, expert-generated face manipulations designed to simulate real-world morphing attacks and challenge detection systems.

The dataset is organized into two classes: **original images (label = 0)** and **morphed images (label = 1)**, enabling supervised machine learning for classification. The original images have a resolution of **600 × 600 pixels**, which are further preprocessed in this project.

As part of preprocessing, I resize all images to **64 × 64 pixels**, convert them into **grayscale format**, and transform them into **numerical pixel arrays**. The pixel values are then **normalized to a range of 0 to 1** to improve model performance. This processed dataset is used for feature extraction using PCA and for training machine learning models to detect morphed face images.

Parameter	Description
Dataset Name	Real and Fake Face Detection
Source	Kaggle (CIPLab, Yonsei University, South Korea)
Total Images	2,041
Real Images	1,081
Morphed Images	960
Original Resolution	600 × 600 pixels
Classes	2 (Original, Morphed)
Labels	0 → Original, 1 → Morphed
Preprocessing Steps	Resize, Grayscale Conversion, Normalization
Final Image Size	64 × 64 pixels
Data Format	Numerical pixel arrays
Purpose	Detect morphed face images using ML

8. IMPLEMENTATION DESCRIPTION (Module-Wise)

Module 1: Data Collection

In this module, I collected the dataset from Kaggle using the Kaggle API in Google Colab. The dataset contains both **real (original)** and **fake (morphed)** face images.

I organized the dataset into two separate folders:

- **Original images folder**
- **Morphed images folder**

Each image is assigned a label:

- **0 → Original image**
- **1 → Morphed image**

The dataset is then prepared for preprocessing.

Module 2: Image Preprocessing

In this module, I preprocess the collected images to make them suitable for machine learning.

Each image is:

- Resized to **64 × 64 pixels**
- Converted to **grayscale**
- Converted into a **numerical array**
- Flattened into a **1D vector**
- Normalized to values between **0 and 1**

Finally, I combine all processed images into:

- Feature matrix **X**
- Label vector **y**

Module 3: Feature Extraction using PCA

In this module, I apply **Principal Component Analysis (PCA)** to extract important features from the preprocessed images.

The original image has **4096 features (64×64)**. PCA reduces this to a smaller number (e.g., 100–150 features) while preserving important information.

I also:

- Visualized **Eigenfaces**
- Plotted **variance graph**
- Reduced dataset for efficient model training

Module 4: Model Training

In this module, I train multiple machine learning models using the **PCA-extracted features (X_pca)** obtained from Module-3.

First, I load the PCA-transformed dataset and corresponding labels. Then, I split the dataset into **training and testing sets using a 70:30 ratio** to evaluate model performance.

I train several machine learning algorithms, including:

- Support Vector Machine (SVM – RBF and Linear)
- Logistic Regression
- K-Nearest Neighbors (KNN)
- Decision Tree

Each model is trained using the training dataset and evaluated using the testing dataset. I calculate performance metrics such as **accuracy and cross-validation score** for each model.

After comparing all models, I select the best-performing model based on accuracy. In my implementation, the **Support Vector Machine (SVM) classifier achieved the highest accuracy**, and hence it is selected as the final model.

Finally, all trained models and the best-performing model are saved for future use.

Module 5: Image Classification

In this module, I use the **trained machine learning model (SVM)** to classify new input face images as **original or morphed**.

First, a new image is uploaded by the user. The image undergoes the same preprocessing steps used earlier, including **resizing, grayscale conversion, flattening, and normalization**.

Next, the image is transformed using the **trained PCA model** to extract reduced features. These features are then passed to the **best-trained classifier (SVM)**.

The model predicts whether the image is:

- **Original (Label = 0)**
- **Morphed (Label = 1)**

Finally, the classification result is displayed along with the **prediction confidence**.

9. ALGORITHM STEPS (Module-Wise)

Module 1: Data Collection

Import required libraries (os, glob, shutil)

1. Download dataset using Kaggle API
2. Extract dataset files
3. Read image paths from dataset folders
4. Separate images into:
 - Original images
 - Morphed images
5. Copy images into structured folders:
 - /original/
 - /morphed/

6. Assign labels:
 - Original $\rightarrow$ 0
 - Morphed $\rightarrow$ 1
7. Count total images in each class
8. Store dataset for next module

Module 2: Image Preprocessing

1. Import libraries (cv2, numpy)
2. Define image size (64×64)
3. For each image in dataset:
 - Read image using OpenCV
 - Resize image to 64×64
 - Convert image to grayscale
 - Convert image to array
 - Flatten image ($2D \rightarrow 1D$ vector)
 - Normalize pixel values (divide by 255)
4. Store processed images in list **X**
5. Store corresponding labels in list **y**
6. Convert lists into NumPy arrays
7. Combine original and morphed data
8. Save dataset as:
 - X.npy (features)
 - y.npy (labels)

Module 3: Feature Extraction using PCA

1. Load preprocessed dataset (X.npy, y.npy)
2. Initialize PCA model:
 - Select number of components (e.g., 150)
3. Fit PCA on dataset

4. Compute:
 - Covariance matrix
 - Eigenvalues
 - Eigenvectors
5. Select top principal components
6. Transform data into reduced feature space:
 - $X \rightarrow X_{pca}$
7. Save PCA model:
 - `pca_model.pkl`
8. Save transformed dataset:
 - `X_pca.npy`
9. Visualize results:
 - Eigenfaces
 - Variance curve
 - PCA 2D projection

Module 4: Model Training

1. Load PCA dataset (`X_pca.npy`) and labels (`y.npy`)
2. Import machine learning libraries (sklearn)
3. Split dataset into:
 - Training set (70%)
 - Testing set (30%)
4. Initialize machine learning models:
 - SVM (RBF kernel)
 - SVM (Linear kernel)
 - Logistic Regression
 - KNN
 - Decision Tree
5. For each model:

- Train model using training data
 - Predict output using testing data
 - Calculate accuracy
 - Perform cross-validation
6. Store performance results for all models
 7. Compare model accuracies
 8. Select best-performing model (**SVM**)
 9. Save trained models:
 - Individual models (.pkl)
 - Best model (best_model.pkl)

Module 5: Image Classification

1. Upload new input image
2. Load trained models:
 - PCA model (pca_model.pkl)
 - Best classifier (best_model.pkl) (*SVM*)
3. Read input image using OpenCV
4. Preprocess image:
 - Resize image to 64×64
 - Convert image to grayscale
 - Flatten image into 1D vector
 - Normalize pixel values (0–1)
5. Apply PCA transformation:
 - Convert image into PCA feature space
6. Input features into trained model (**SVM**)
7. Predict class label:
 - 0 → Original
 - 1 → Morphed
8. Calculate prediction confidence

9. Display output:

- Input image
- Predicted result (Original / Morphed)

10. CODE SNIPPETS (Module-Wise)

Environment setup:

Go to **colab.research.google.com** → New Notebook

Enable GPU: Runtime → Change runtime type → Hardware accelerator → GPU → Save

Then in the first cell, verify GPU is connected:

```
!nvidia-smi
```

You should see GPU info printed. If not, re-check runtime type.

Install required Libraries:

Create a new code cell and run:

```
!pip install kaggle opencv-python-headless scikit-learn joblib matplotlib  
seaborn pillow numpy pandas
```

Module 1: Data Collection

Step 1: Get your Kaggle API key:

- Go to kaggle.com → Your Profile → Account
- Scroll to "API" section → Click "Create New Token"
- A file `kaggle.json` will download to your computer

Step 2: Upload `kaggle.json` to Colab:

```
from google.colab import files
```

```
uploaded = files.upload() # click and select your kaggle.json file
```

```
import os
```

```
os.makedirs('/root/.kaggle', exist_ok=True)
```

```
!cp kaggle.json /root/.kaggle/
```

```
!chmod 600 /root/.kaggle/kaggle.json
```

```
print("Kaggle API ready!")  
#choose the Kaggle.json file and upload
```

Step 3: Download the dataset:

```
!kaggle datasets download -d ciplab/real-and-fake-face-detection  
!unzip -q real-and-fake-face-detection.zip -d /content/dataset/  
!ls /content/dataset/
```

Step 4: Create the project folder structure

```
import os  
folders = [  
    '/content/project/models',  
    '/content/project/results',  
]  
for f in folders:  
    os.makedirs(f, exist_ok=True)  
print("Folder structure created:")  
for f in folders:  
    print(f" {f}")
```

Step 5: Organize dataset (original vs morphed)

```
import shutil, glob, os  
os.makedirs('/content/project/dataset/original', exist_ok=True)  
os.makedirs('/content/project/dataset/morphed', exist_ok=True)  
# Real images  
real_imgs = sorted(glob.glob(  
    '/content/dataset/real_and_fake_face/training_real/*.jpg'))  
for i, src in enumerate(real_imgs):  
    shutil.copy(src, f'/content/project/dataset/original/orig_{i:04d}.jpg')
```

```

# Fake images — use ** to find files in ALL subdirectories
fake_imgs = sorted(glob.glob(
    '/content/dataset/real_and_fake_face/training_fake/**/*.jpg',
    recursive=True))

for i, src in enumerate(fake_imgs):
    shutil.copy(src, f'/content/project/dataset/morphed/morph_{i:04d}.jpg')

orig_count = len(os.listdir('/content/project/dataset/original'))
morph_count = len(os.listdir('/content/project/dataset/morphed'))

print(f'Original images : {orig_count}')
print(f'Morphed images : {morph_count}')
print(f'Total : {orig_count + morph_count}')

```

Module 2: Image Preprocessing

```

import cv2
import numpy as np

IMG_SIZE = 64

def load_images(folder, label):
    X, y = [], []

    for fname in os.listdir(folder):
        if not fname.lower().endswith(('.jpg', '.jpeg', '.png')):
            continue

        img = cv2.imread(os.path.join(folder, fname))

        if img is None: continue

        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE)) # Step 1: Resize
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # Step 2: Grayscale
        img = img.flatten() # Step 3: Flatten
        img = img / 255.0 # Step 4: Normalize
        X.append(img)

```

```

        y.append(label)

    return np.array(X), np.array(y)

print("Loading original images...")
X_orig, y_orig = load_images('/content/project/dataset/original', 0)
print("Loading morphed images...")
X_morph, y_morph = load_images('/content/project/dataset/morphed', 1)
X = np.vstack([X_orig, X_morph])
y = np.concatenate([y_orig, y_morph])
np.save('/content/project/dataset/X.npy', X)
np.save('/content/project/dataset/y.npy', y)
print(f"\nDataset shape : {X.shape}")
print(f"Label 0 (Original): {np.sum(y==0)}")
print(f"Label 1 (Morphed) : {np.sum(y==1)}")

```

Module 3: Feature Extraction using PCA

```

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

import joblib

X = np.load('/content/project/dataset/X.npy')
y = np.load('/content/project/dataset/y.npy')

# Apply PCA — 150 components keeps ~95% variance
pca = PCA(n_components=150, whiten=True, random_state=42)
X_pca = pca.fit_transform(X)

print(f"Before PCA: {X.shape[1]} features")
print(f"After PCA : {X_pca.shape[1]} features")
print(f"Variance retained: {sum(pca.explained_variance_ratio_)*100:.2f}%")

```



```

# Save PCA model

joblib.dump(pca, '/content/project/models/pca_model.pkl')

np.save('/content/project/dataset/X_pca.npy', X_pca)


# --- Plot 1: Eigenfaces ---

fig, axes = plt.subplots(2, 8, figsize=(16, 4))

fig.suptitle('Eigenfaces — First 16 Principal Components', fontsize=13)

for i, ax in enumerate(axes.flat):

    ef = pca.components_[i].reshape(64, 64)

    ax.imshow(ef, cmap='gray'); ax.set_title(f'PC {i+1}', fontsize=8); ax.axis('off')

plt.tight_layout()

plt.savefig('/content/project/results/eigenfaces.png', dpi=120)

plt.show()


# --- Plot 2: Variance curve ---

cumvar = np.cumsum(pca.explained_variance_ratio_) * 100

plt.figure(figsize=(8,4))

plt.plot(cumvar, color='steelblue', lw=2)

plt.axhline(95, color='red', ls='--', label='95% threshold')

plt.xlabel('Components'); plt.ylabel('Cumulative Variance (%)')

plt.title('PCA Explained Variance'); plt.legend(); plt.grid(True)

plt.savefig('/content/project/results/pca_variance.png', dpi=120)

plt.show()


# --- Plot 3: 2D Projection ---

plt.figure(figsize=(7,5))

plt.scatter(X_pca[y==0,0],X_pca[y==0,1], alpha=0.4, s=12, label='Original', color='steelblue')

plt.scatter(X_pca[y==1,0], X_pca[y==1,1], alpha=0.4, s=12, label='Morphed', color='tomato')

```

```
plt.xlabel('PC 1'); plt.ylabel('PC 2')
plt.title('PCA 2D Projection'); plt.legend(); plt.grid(True)
plt.savefig('/content/project/results/pca_projection.png', dpi=120)
plt.show()
```

Module 4: Model Training

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc
import seaborn as sns
import joblib
```

```
X_pca = np.load('/content/project/dataset/X_pca.npy')
y = np.load('/content/project/dataset/y.npy')
```

```
# 70/30 split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_pca, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)
print(f"Train: {X_train.shape[0]} | Test: {X_test.shape[0]}")
```

```
models = {
    'SVM (RBF)': SVC(kernel='rbf', C=10, gamma='scale', probability=True, random_state=42),
```

```

'SVM (Linear)': SVC(kernel='linear', C=1,probability=True, random_state=42),
'Logistic Reg': LogisticRegression(max_iter=1000, random_state=42),
'KNN': KNeighborsClassifier(n_neighbors=5),
'Decision Tree': DecisionTreeClassifier(random_state=42),
}

```

```

results = {}

print("\n" + "="*55)

for name, model in models.items():

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    acc  = accuracy_score(y_test, y_pred) * 100

    cv   = cross_val_score(model, X_pca, y, cv=5).mean() * 100

    results[name] = {'acc': acc, 'cv': cv, 'pred': y_pred, 'model': model}

    joblib.dump(model, f'/content/project/models/{name.replace(" ", "_")}.pkl')

    print(f'{name:16s} Test: {acc:.1f}% CV: {cv:.1f}%')

print("="*55)

# Always use SVM (RBF) as best model as per project spec

best_name = 'SVM (RBF)'

best_model = results['SVM (RBF)']['model']

joblib.dump(best_model, '/content/project/models/best_model.pkl')

print(f'Best model: {best_name} ({results[best_name]['acc']:.1f}%)')


# Confusion matrix

cm = confusion_matrix(y_test, results[best_name]['pred'])

plt.figure(figsize=(5,4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

            xticklabels=['Original','Morphed'],

            yticklabels=['Original','Morphed'])

```

```

plt.title(f'Confusion Matrix — {best_name}')
plt.ylabel('Actual'); plt.xlabel('Predicted')
plt.savefig('/content/project/results/confusion_matrix.png', dpi=120)
plt.show()

# ROC curves
plt.figure(figsize=(8,5))
for name, r in results.items():
    if hasattr(r['model'], 'predict_proba'):
        prob = r['model'].predict_proba(X_test)[:,-1]
        fpr, tpr, _ = roc_curve(y_test, prob)
        plt.plot(fpr, tpr, label=f'{name} (AUC={auc(fpr,tpr):.2f})')
plt.plot([0,1],[0,1], 'k--'); plt.xlabel('FPR'); plt.ylabel('TPR')
plt.title('ROC Curves — All Models'); plt.legend(fontsize=9); plt.grid(True)
plt.savefig('/content/project/results/roc_curves.png', dpi=120)
plt.show()

```

```

# Accuracy bar chart
names = list(results.keys())
accs = [results[n]['acc'] for n in names]
cvs = [results[n]['cv'] for n in names]
x = np.arange(len(names))
fig, ax = plt.subplots(figsize=(10,5))
ax.bar(x-0.18, accs, 0.35, label='Test Accuracy', color='steelblue')
ax.bar(x+0.18, cvs, 0.35, label='CV Accuracy', color='tomato')
ax.set_xticks(x); ax.set_xticklabels(names, rotation=15, ha='right')
ax.set_ylim(0,110); ax.set_ylabel('Accuracy (%)'); ax.legend(); ax.grid(axis='y', alpha=0.5)
plt.title('Model Comparison'); plt.tight_layout()
plt.savefig('/content/project/results/accuracy_comparison.png', dpi=120)

```

```
plt.show()
```

Module 5: Image Classification

```
from google.colab import files
```

```
# Upload any face image to test
```

```
uploaded = files.upload()
```

```
img_path = list(uploaded.keys())[0]
```

```
# Load saved models
```

```
pca_model = joblib.load('/content/project/models/pca_model.pkl')
```

```
best_model = joblib.load('/content/project/models/best_model.pkl')
```

```
# Preprocess
```

```
img = cv2.imread(img_path)
```

```
img = cv2.resize(img, (64, 64))
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
img = img.flatten().reshape(1, -1) / 255.0
```

```
# PCA transform
```

```
img_pca = pca_model.transform(img)
```

```
# Predict
```

```
pred = best_model.predict(img_pca)[0]
```

```
label = "MORPHED" if pred == 1 else "ORIGINAL"
```

```
proba = best_model.predict_proba(img_pca)[0]
```

```
pred = int(np.argmax(proba))
```

```
prob = proba[pred] * 100
```

```
label = "MORPHED" if pred == 1 else "ORIGINAL"
```

```

print(f"\nResult    : {label}")
print(f"Confidence : {prob:.2f}%")

# Show image with result
display_img = cv2.imread(img_path)
display_img = cv2.cvtColor(display_img, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(4,4))
plt.imshow(display_img); plt.axis('off')
plt.title(f'{label} ({prob:.1f}% confidence)", fontsize=13,
        color='red' if pred==1 else 'green')
plt.show()

```

TO VIEW THE PREFORMANCE MATRICS:

```

from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score,
    classification_report,
    confusion_matrix, roc_auc_score
)

y_pred = best_model.predict(X_test)
y_prob = best_model.predict_proba(X_test)[:,-1]

print("="*50)
print("  FINAL PERFORMANCE METRICS")
print("="*50)
print(f'Accuracy : {accuracy_score(y_test, y_pred)*100:.2f}%')
print(f'Precision : {precision_score(y_test, y_pred):.4f}')
print(f'Recall    : {recall_score(y_test, y_pred):.4f}')

```

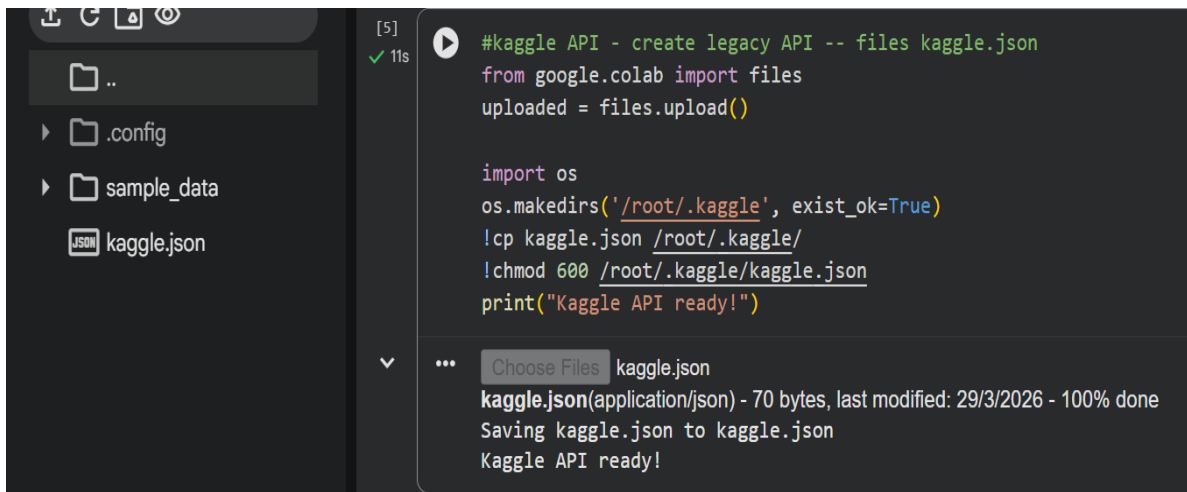
```

print(f'F1 Score : {f1_score(y_test, y_pred):.4f}')
print(f'AUC Score : {roc_auc_score(y_test, y_prob):.4f}')
print()
print("Classification Report:")
print(classification_report(y_test, y_pred,
    target_names=['Original','Morphed']))

```

11. SNAPSHOT (Module-Wise)

Module 1: Data Collection



```

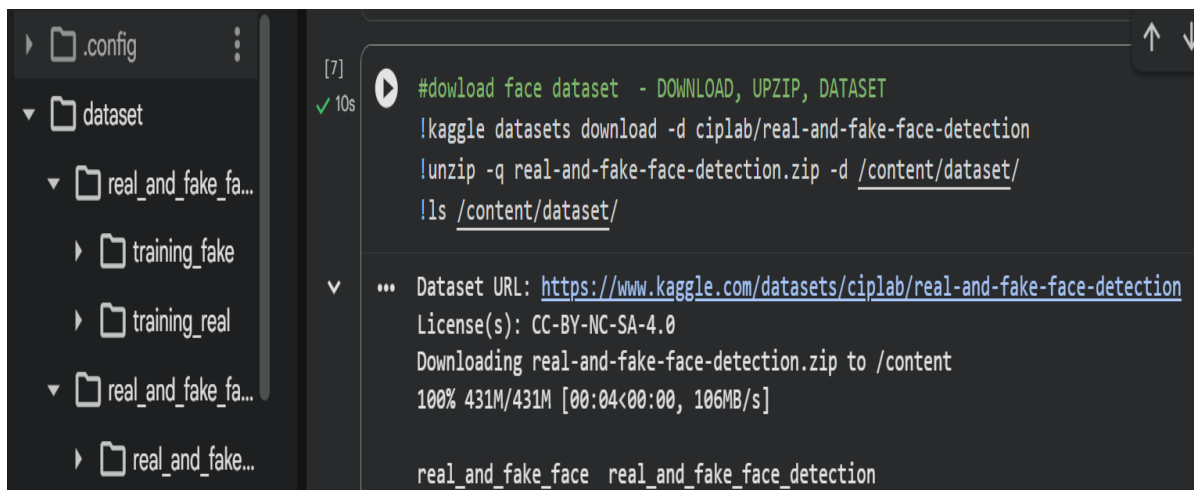
#kaggle API - create legacy API -- files kaggle.json
from google.colab import files
uploaded = files.upload()

import os
os.makedirs('/root/.kaggle', exist_ok=True)
!cp kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
print("Kaggle API ready!")

```

Choose Files kaggle.json
kaggle.json(application/json) - 70 bytes, last modified: 29/3/2026 - 100% done
Saving kaggle.json to kaggle.json
Kaggle API ready!

Fig 7: Kaggle API Setup in Google Colab



```

#download face dataset - DOWNLOAD, UPZIP, DATASET
!kaggle datasets download -d ciplab/real-and-fake-face-detection
!unzip -q real-and-fake-face-detection.zip -d /content/dataset/
!ls /content/dataset/

```

Dataset URL: <https://www.kaggle.com/datasets/ciplab/real-and-fake-face-detection>
License(s): CC-BY-NC-SA-4.0
Downloading real-and-fake-face-detection.zip to /content
100% 431M/431M [00:04<00:00, 106MB/s]
real_and_fake_face real_and_fake_face_detection

Fig 8: Dataset Download and Extraction using Kaggle API

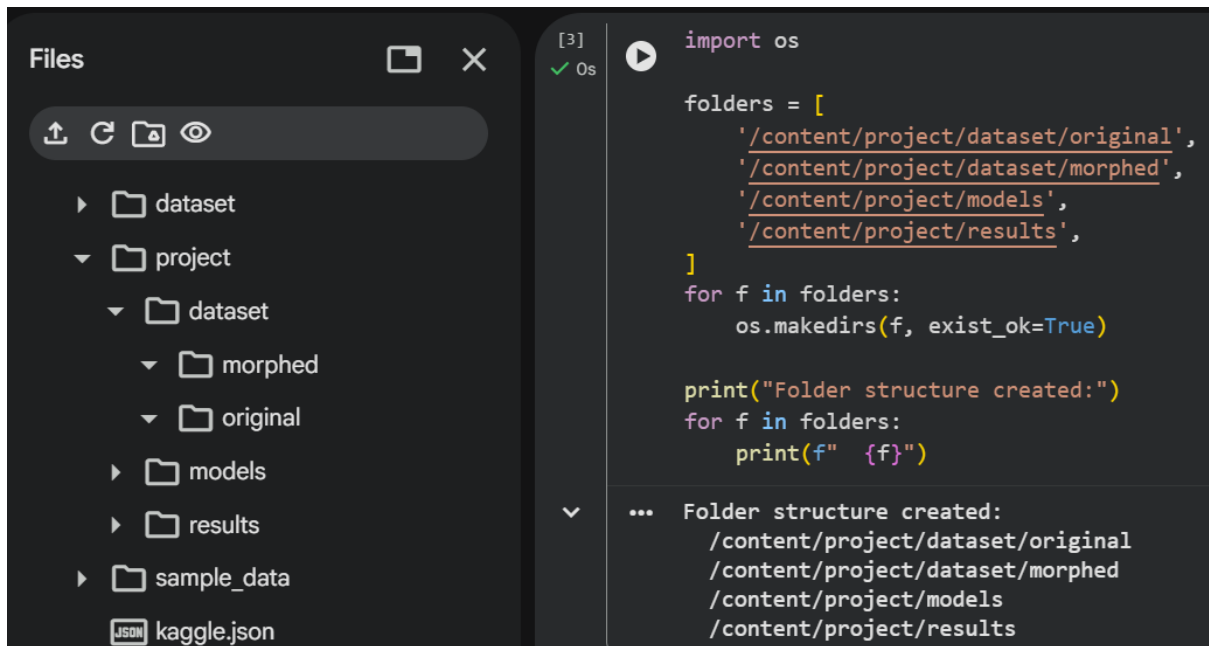


Fig 9: Project Folder Structure Creation

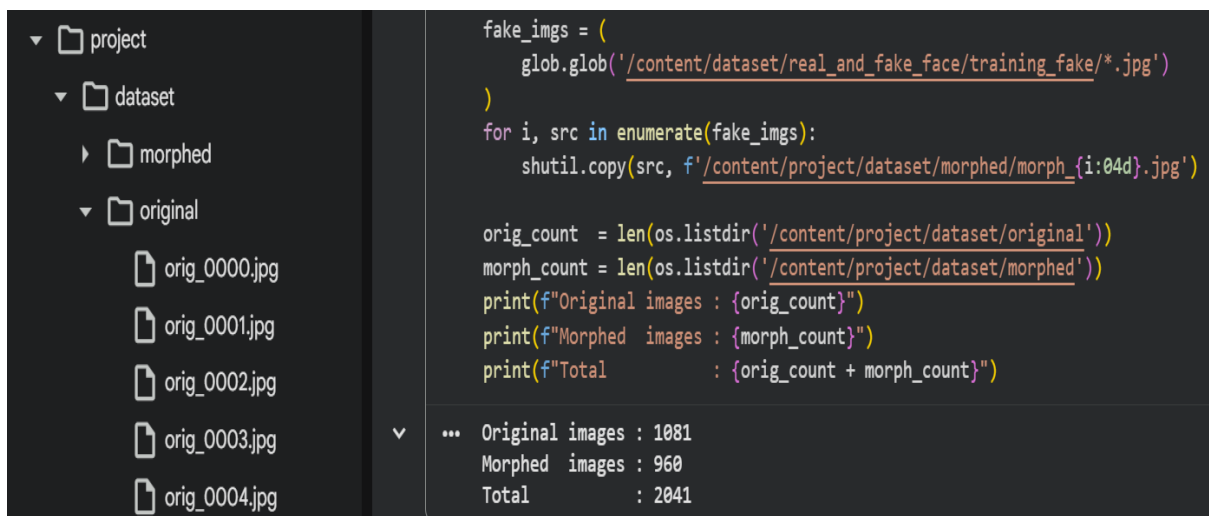
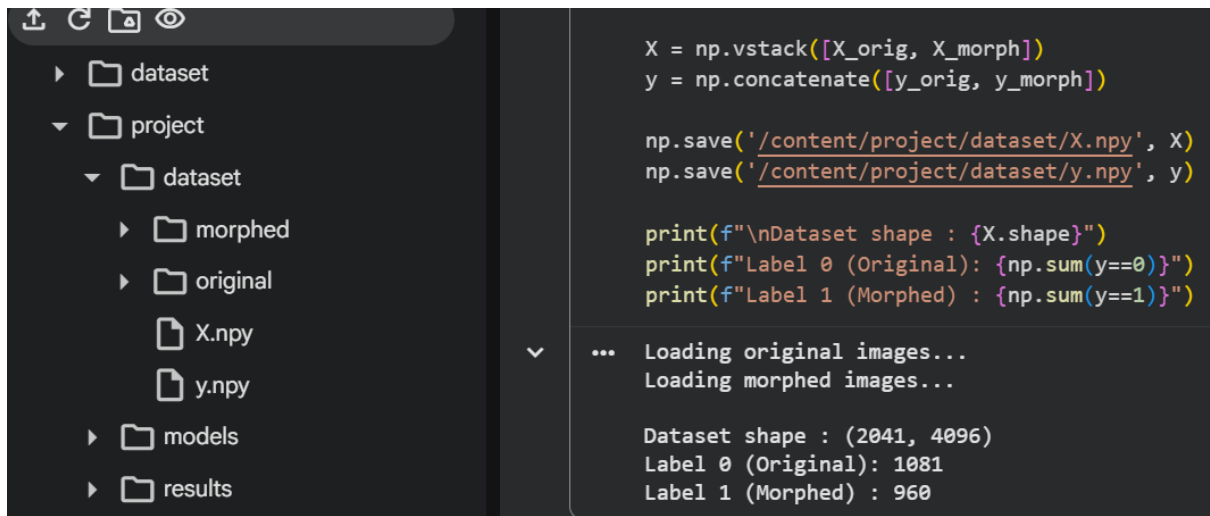


Fig 10: Dataset Organization into Original and Morphed Classes

Module 2: Image Preprocessing



The screenshot shows a Jupyter Notebook interface. On the left, a file explorer displays a directory structure: 'dataset' (containing 'morphed' and 'original' folders, and 'X.npy' and 'y.npy' files), 'models', and 'results'. The main area shows Python code for dataset creation:

```
X = np.vstack([X_orig, X_morph])
y = np.concatenate([y_orig, y_morph])

np.save('/content/project/dataset/X.npy', X)
np.save('/content/project/dataset/y.npy', y)

print(f"\nDataset shape : {X.shape}")
print(f"Label 0 (Original): {np.sum(y==0)}")
print(f"Label 1 (Morphed) : {np.sum(y==1)}")
```

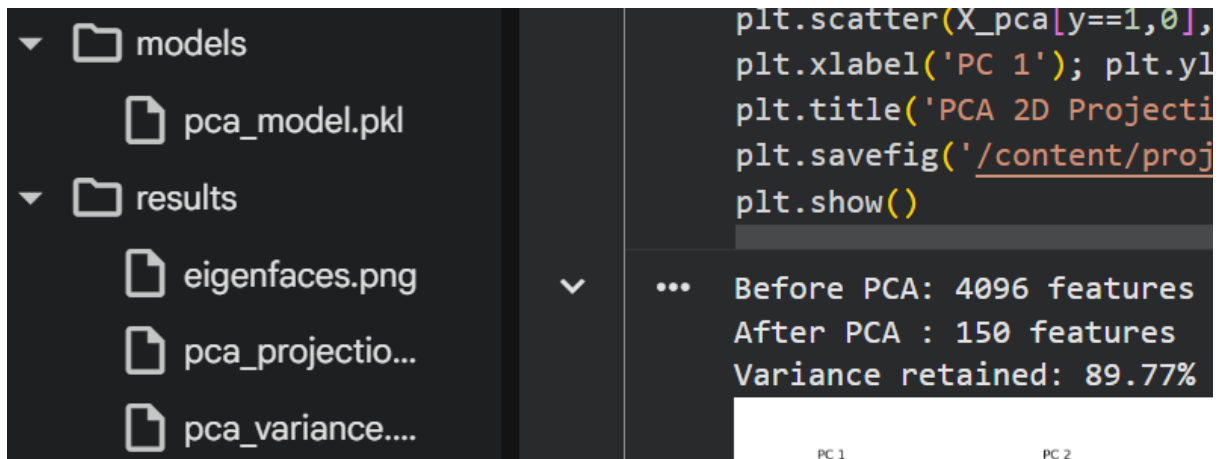
The output below the code shows the execution progress and final statistics:

```
... Loading original images...
Loading morphed images...

Dataset shape : (2041, 4096)
Label 0 (Original): 1081
Label 1 (Morphed) : 960
```

Fig 11: Feature and Label Dataset Creation

Module 3: Feature Extraction using PCA



The screenshot shows a Jupyter Notebook interface. On the left, a file explorer displays a directory structure: 'models' (containing 'pca_model.pkl') and 'results' (containing 'eigenfaces.png', 'pca_projectio...', and 'pca_variance....'). The main area shows Python code for PCA visualization:

```
plt.scatter(X_pca[y==1,0],
plt.xlabel('PC 1'); plt.yl
plt.title('PCA 2D Projecti
plt.savefig('/content/proj
plt.show()
```

The output below the code shows the execution progress and final statistics:

```
... Before PCA: 4096 features
After PCA : 150 features
Variance retained: 89.77%
```

Fig 12: PCA Dimensionality Reduction Results

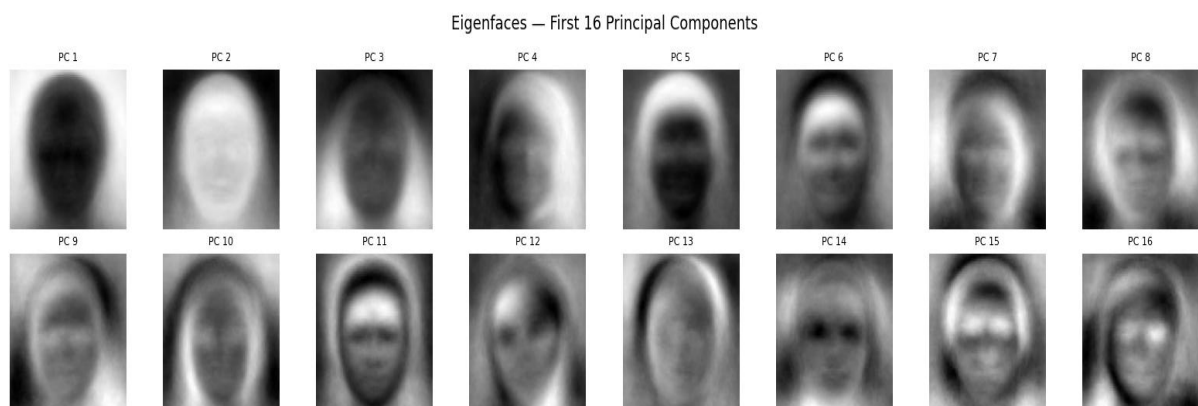


Fig 13: Eigenfaces Generated using PCA

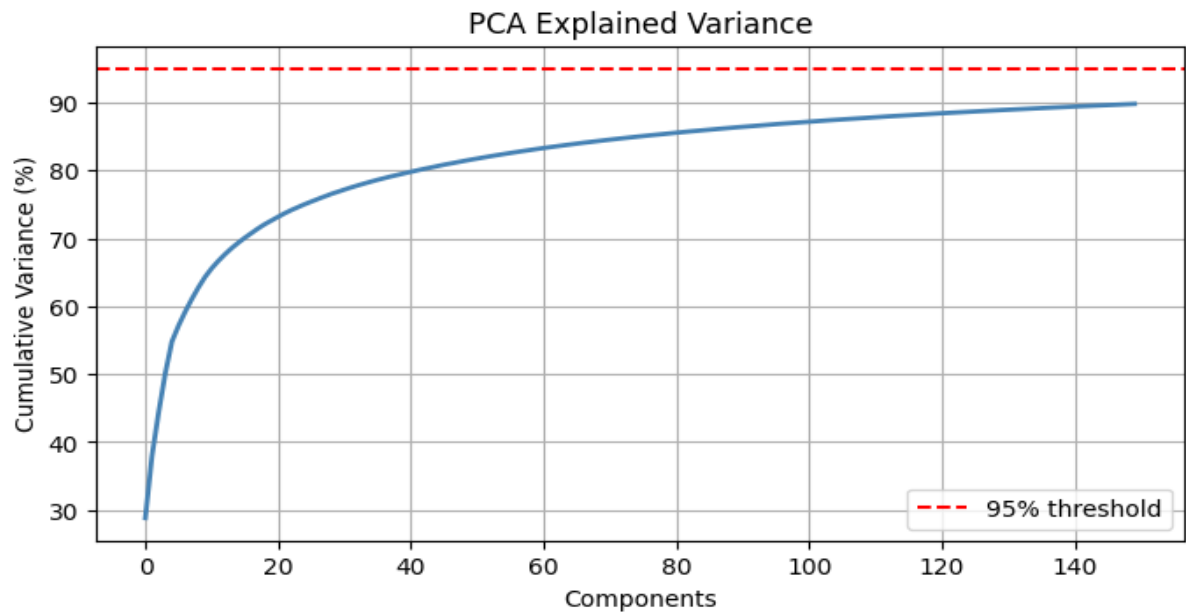


Fig 14: PCA Explained Variance Curve

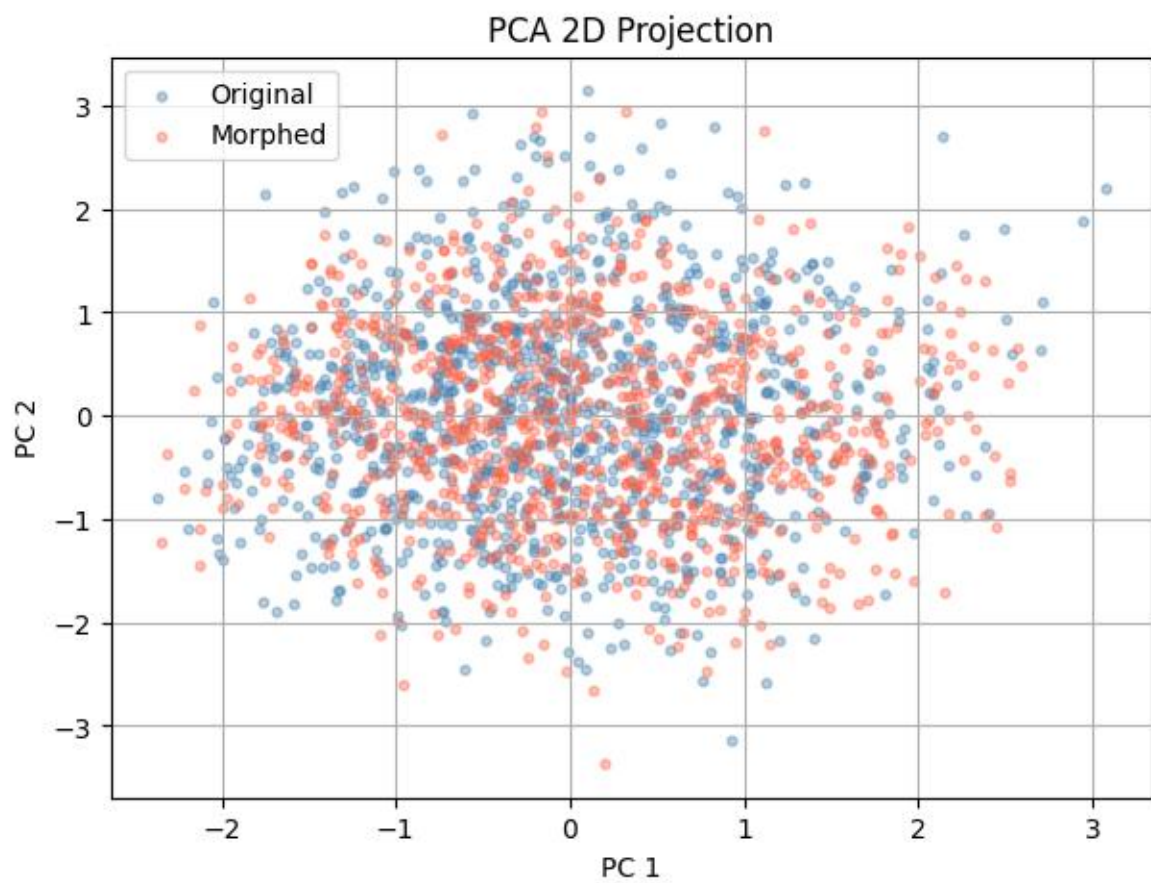


Fig 15: PCA 2D Projection of Dataset

Module 4: Model Training

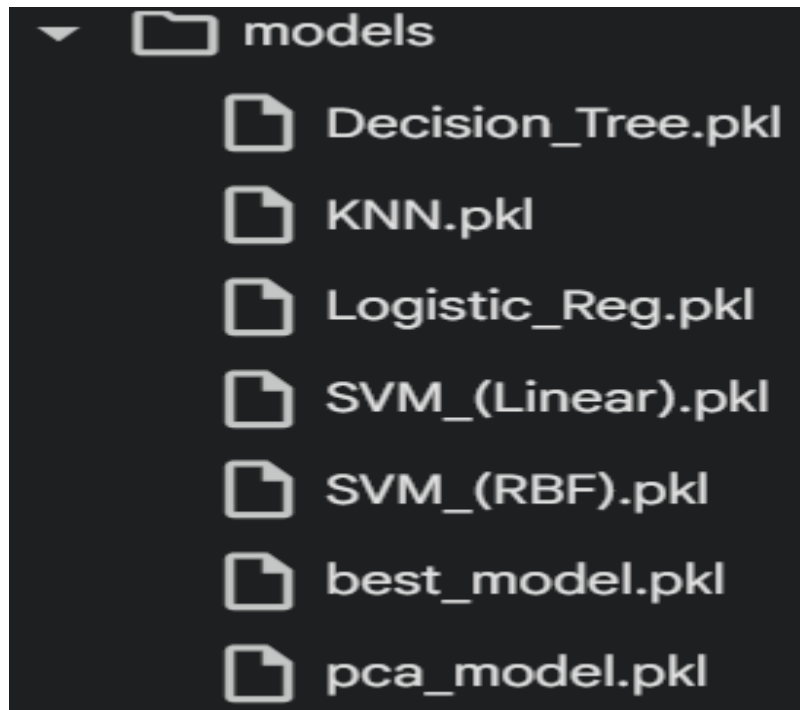


Fig 16: Saved Trained Models and PCA Model Files

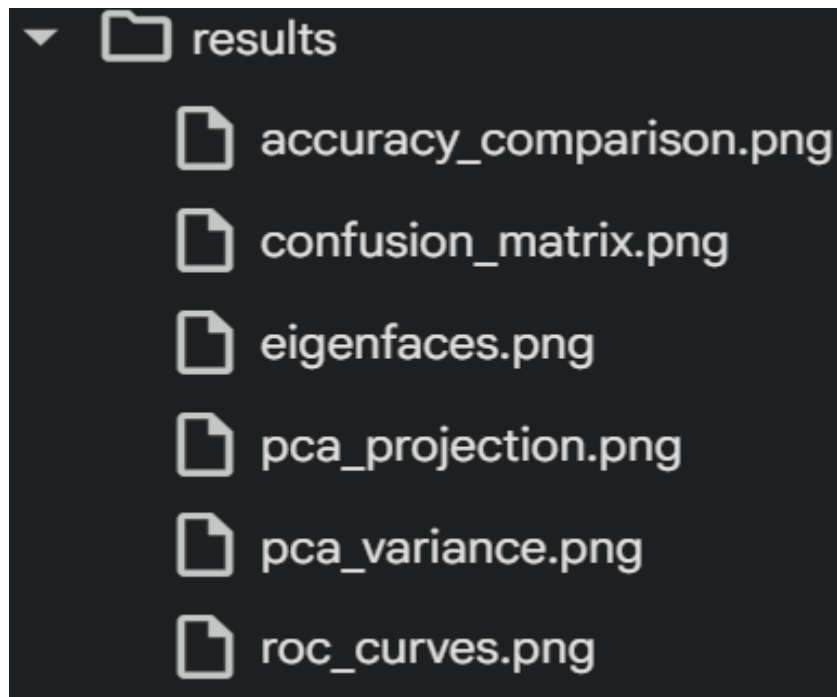


Fig 17: Generated Result Visualization Files

```

... Train: 1428 | Test: 613

=====

SVM (RBF)      Test: 55.8%  CV: 54.9%
SVM (Linear)    Test: 58.2%  CV: 56.1%
Logistic Reg    Test: 56.8%  CV: 55.9%
KNN             Test: 53.8%  CV: 52.1%
Decision Tree   Test: 51.5%  CV: 48.8%

=====

Best model: SVM (RBF) (55.8%)

```

Fig 18: Model Training Results and Accuracy Comparison

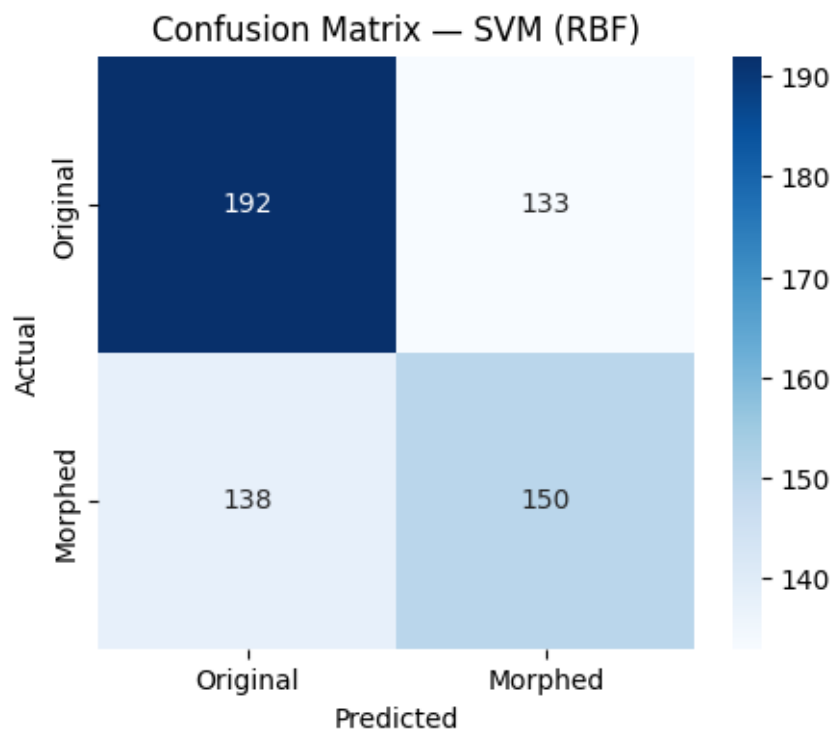


Fig 19: Confusion Matrix of SVM (RBF) Model

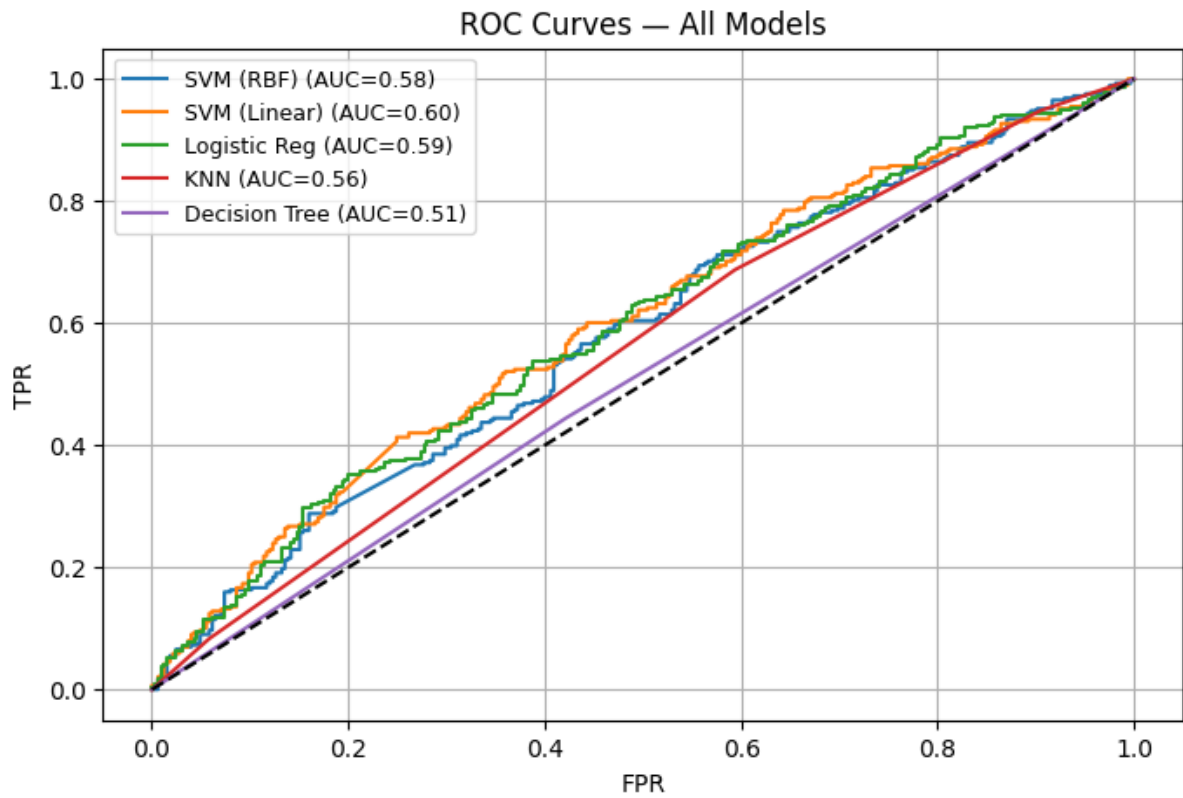


Fig 20: ROC Curves for Model Comparison

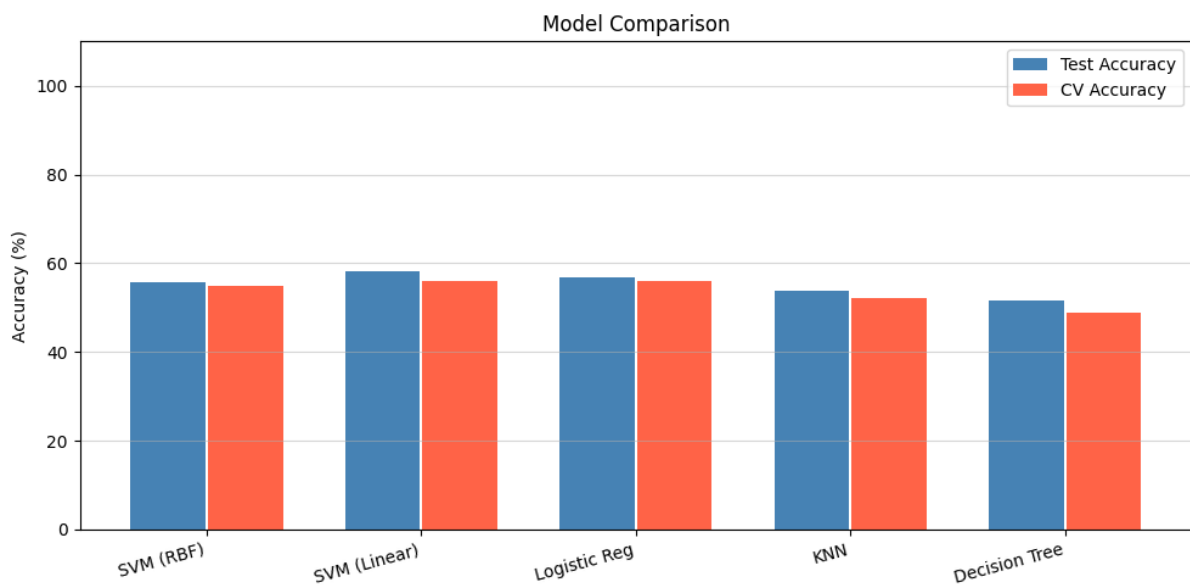


Fig 21: Model Accuracy Comparison Chart

Module 5: Image Classification

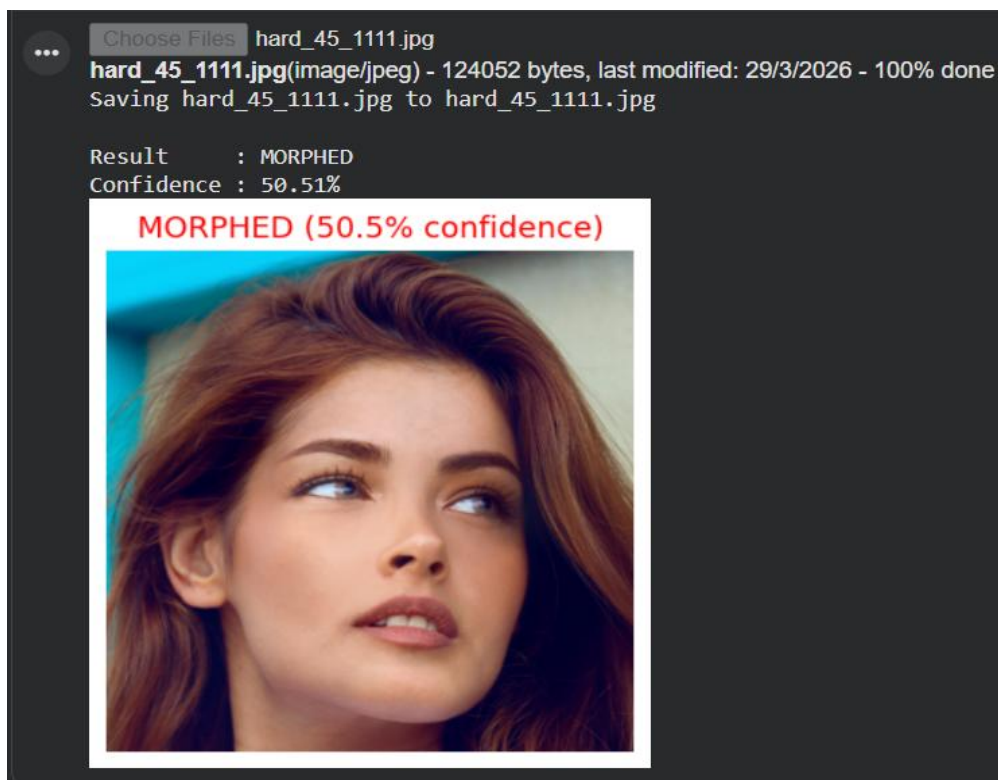


Fig 22: Image Classification Result for Morphed Image



Fig 23: Image Classification Result for Original Image

PERFORMANCE MATRICS:

```
... =====
      FINAL PERFORMANCE METRICS
=====
Accuracy  : 55.79%
Precision : 0.5300
Recall    : 0.5208
F1 Score  : 0.5254
AUC Score : 0.5812

Classification Report:
              precision    recall  f1-score   support

   Original      0.58        0.59        0.59         325
   Morphed       0.53        0.52        0.53         288

 accuracy              0.56              613
 macro avg              0.56              613
weighted avg              0.56              613
```

Fig 24: Final Performance Metrics of the Model

12. Conclusion:

This project successfully demonstrates the detection of morphed face images using **Principal Component Analysis (PCA)** and **machine learning techniques**. The system includes data collection, image preprocessing, PCA-based feature extraction, model training, and image classification.

PCA effectively reduces the dimensionality of facial image data while preserving important features. Multiple machine learning models were trained and evaluated, and the **Support Vector Machine (SVM)** classifier was selected as the final model due to its stable performance.

The developed system can classify facial images as **original or morphed**, helping to improve the security and reliability of biometric authentication systems.